Lecture 1. Overview of Programming Paradigms

Programming Paradigms, Fall 2026 — 26 Aug 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. About This Course
2. Programming Paradigms Overview
3. Programming Paradigms and Concepts in This Course
4. About Functional Programming
5. Introduction to λ-calculus

About This Course

Why Learn This When AI Writes the Code?

It is a fair question.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code. You define correct, and you check it.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code. You define correct, and you check it.
2. Choosing how to model a problem is not code. It is the decision made before any code exists, and it is what this course is about.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code. You define correct, and you check it.
2. Choosing how to model a problem is not code. It is the decision made before any code exists, and it is what this course is about.
3. The failures that survive review are the semantic ones.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code. You define correct, and you check it.
2. Choosing how to model a problem is not code. It is the decision made before any code exists, and it is what this course is about.
3. The failures that survive review are the semantic ones. Evaluation order, effects, laziness, search order.

Why Learn This When AI Writes the Code?

It is a fair question.

1. An AI assistant produces code. You define correct, and you check it.
2. Choosing how to model a problem is not code. It is the decision made before any code exists, and it is what this course is about.
3. The failures that survive review are the semantic ones. Evaluation order, effects, laziness, search order. You cannot review what you cannot read.

Question

When you last accepted a suggestion, what did you check first?

AI Assistants Are Weakest Exactly Here

FPBench evaluates language models on 721 functional programming tasks[1]:

Language	GPT-4o	gap to Java
Java	43.7%	—
OCaml	36.2%	−7.5 points
Haskell	27.2%	−16.5 points

[1] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

AI Assistants Are Weakest Exactly Here

FPBench evaluates language models on 721 functional programming tasks[1]:

Language	GPT-4o	gap to Java
Java	43.7%	—
OCaml	36.2%	−7.5 points
Haskell	27.2%	−16.5 points

On GPT-5 the Haskell gap grows to about 18 points.

[1] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

AI Assistants Are Weakest Exactly Here

FPBench evaluates language models on 721 functional programming tasks[1]:

Language	GPT-4o	gap to Java
Java	43.7%	—
OCaml	36.2%	−7.5 points
Haskell	27.2%	−16.5 points

On GPT-5 the Haskell gap grows to about 18 points.

The reason is mundane: far more Python and Java in the data.

[1] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

And They Write Haskell as if It Were Java

In the same study, the code that models produced carried imperative patterns at a high rate[2]:

Language	imperative style (GPT-5)
Scala	94%
Haskell	88%
OCaml	80%

[2] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

And They Write Haskell as if It Were Java

In the same study, the code that models produced carried imperative patterns at a high rate[2]:

Language	imperative style (GPT-5)
Scala	94%
Haskell	88%
OCaml	80%

The code compiles, passes the tests, and is not the program you wanted.

[2] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

And They Write Haskell as if It Were Java

In the same study, the code that models produced carried imperative patterns at a high rate[2]:

Language	imperative style (GPT-5)
Scala	94%
Haskell	88%
OCaml	80%

The code compiles, passes the tests, and is not the program you wanted.

Pitfall

> Knowing what idiomatic looks like is learned, not prompted.

[2] Nguyet-Anh H. Lang et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.***

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

- It is a **functional logic** language: functions, but also logic variables, unification, and expressions that yield many results.

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

- It is a **functional logic** language: functions, but also logic variables, unification, and expressions that yield many results.
- Live in Fortnite's editor since 2023; the language of Unreal Engine 6.

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

- It is a **functional logic** language: functions, but also logic variables, unification, and expressions that yield many results.
- Live in Fortnite's editor since 2023; the language of Unreal Engine 6.
- Its semantics is given by a small core calculus with a rewrite semantics, in the same style as the λ-calculus we start today.

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

- It is a **functional logic** language: functions, but also logic variables, unification, and expressions that yield many results.
- Live in Fortnite's editor since 2023; the language of Unreal Engine 6.
- Its semantics is given by a small core calculus with a rewrite semantics, in the same style as the λ-calculus we start today.

The ingredients: {···} functional, logic, a λ-calculus core.

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

Paradigms Are Not Settled History

Verse is a new language from Epic Games.
It was designed by Tim Sweeney and Simon Peyton Jones[3].

- It is a **functional logic** language: functions, but also logic variables, unification, and expressions that yield many results.
- Live in Fortnite's editor since 2023; the language of Unreal Engine 6.
- Its semantics is given by a small core calculus with a rewrite semantics, in the same style as the λ-calculus we start today.

The ingredients: {···} functional, logic, a λ-calculus core.

That is this course.

[3] Lennart Augustsson et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*.

What is this course about?

In this course, we will

1. Learn different styles of programming, and become better engineers!
2. Look past the syntax, and care about concepts and semantics!
3. Compare how languages support different styles of programming.

Who Is Teaching This

Nikolai Kudasov, Lab of Programming Languages and Compilers.

Three things I work on, each a part of this course from the other side:

- **Syntax with binders** — representing λ-terms so that substitution cannot capture a variable. We meet that in an hour; *Free Foil*[4] takes it seriously.
- **Relational programming** — `typedKanren`, a typed miniKanren in Haskell.
- **Proof assistants and type theory** — *Rzk*, for simplicial type theory.

[4] Nikolai Kudasov et al. (2024). ***Free Foil: Generating Efficient and Scope-Safe Abstract Syntax.***

Who Is Teaching This

Nikolai Kudasov, Lab of Programming Languages and Compilers.

Three things I work on, each a part of this course from the other side:

- {···} **Syntax with binders** — representing λ-terms so that substitution cannot capture a variable. We meet that in an hour; *Free Foil*[4] takes it seriously.
- **Relational programming** — `typedKanren`, a typed miniKanren in Haskell.
- **Proof assistants and type theory** — *Rzk*, for simplicial type theory.

→ **Come and work on this**

> Open projects and thesis topics in all three.

[4] Nikolai Kudasov et al. (2024). ***Free Foil: Generating Efficient and Scope-Safe Abstract Syntax.***

The Course Team



Mahmoud Naderi

room 413 @Slant14

- PhD student: formal verification of neutral-atom quantum computers
- Full-time Java lover and ambassador
- Ask about compilers, languages, and graph theory. Not about AI.



Anatoliy Baskakov

by arrangement @Nihon_V

- Master's graduate: functional programming and compilers
- Keen on logical puzzles, which is three quarters of this course
- Ask about any of it.

Course Topics

The course consists of the following parts:

Course Topics

The course consists of the following parts:

(1 topic) Overview of paradigms, and a crash course in λ-calculus

Course Topics

The course consists of the following parts:

- (1 topic) Overview of paradigms, and a crash course in λ-calculus
- (4 topics) Functional programming in Racket

Course Topics

The course consists of the following parts:

- (1 topic) Overview of paradigms, and a crash course in λ-calculus
- (4 topics) Functional programming in Racket
- (5 topics) Typed functional programming in Haskell

Course Topics

The course consists of the following parts:

- (1 topic) Overview of paradigms, and a crash course in λ-calculus
- (4 topics) Functional programming in Racket
- (5 topics) Typed functional programming in Haskell
- (4 topics) Relational programming in miniKanren

Course Outline (1 of 2)

Topic	Date	Lecture
λ-calculus		
1	26 Aug	Overview of paradigms; λ-calculus
Racket		dynamically typed functional programming
2	2 Sep	Expressions, pairs, lists; the substitution model
3	9 Sep	Higher-order functions and lists
4	16 Sep	Generators, comprehensions; own datatypes
5	23 Sep	Closures; functional programming in Python and JS

Project 1

Course Outline (2 of 2)

Topic	Date	Lecture	
Haskell		statically typed functional programming	
6	30 Sep	Basics of functional programming in Haskell	Project 2
7	7 Oct	Higher-order functions; algebraic datatypes	
8	14 Oct	Input and output in Haskell	
9	21 Oct	Lazy evaluation in Haskell	
10	28 Oct	Wholemeal programming; typed FP elsewhere	
miniKanren		relational programming	
4 Nov — no class, public holiday			
11	11 Nov	Introduction to miniKanren	Project 3
12	18 Nov	Symbolic constraints in miniKanren	
13	25 Nov	Implementing μKanren in Racket	
14	2 Dec	Implementing miniKanren in Haskell	

Course Outline (2 of 2)



Topic	Date	Lecture	
Haskell		statically typed functional programming	
6	30 Sep	Basics of functional programming in Haskell	Project 2
7	7 Oct	Higher-order functions; algebraic datatypes	
8	14 Oct	Input and output in Haskell	
9	21 Oct	Lazy evaluation in Haskell	
10	28 Oct	Wholemeal programming; typed FP elsewhere	
miniKanren		relational programming	
4 Nov — no class, public holiday			
11	11 Nov	Introduction to miniKanren	Project 3
12	18 Nov	Symbolic constraints in miniKanren	
13	25 Nov	Implementing μKanren in Racket	
14	2 Dec	Implementing miniKanren in Haskell	

A problem set for most topics, and a worksheet in every lab.

Assessment and Grading

30% — 3 mini projects, in Racket, Haskell, and miniKanren

28% — 14 in-lab worksheets

20% — 10 problem sets

15% — oral exam

7% — lecture quizzes, on Moodle, at the end of each lecture

Assessment and Grading

30% — 3 mini projects, in Racket, Haskell, and miniKanren

28% — 14 in-lab worksheets

20% — 10 problem sets

15% — oral exam

7% — lecture quizzes, on Moodle, at the end of each lecture

$A \geq 90\%$ $B \geq 75\%$ $C \geq 60\%$

Assessment and Grading

30% — 3 mini projects, in Racket, Haskell, and miniKanren

28% — 14 in-lab worksheets

20% — 10 problem sets

15% — oral exam

7% — lecture quizzes, on Moodle, at the end of each lecture

$A \geq 90\%$ $B \geq 75\%$ $C \geq 60\%$

Of the 14 worksheets, your **best 11** count: three missed labs cost nothing.

When an Activity Counts as Done

✔ **done** is a technical term in this course.
Every threshold on the next slide is stated in it.

An **in-lab worksheet** is ✔ **done** at **4 out of 10**.

A **problem set** is ✔ **done** when

- it was properly submitted,
- it is marked **60%** or more, and
- you were in the lab to defend it, whether or not you were drawn.

A **mini project** is ✔ **done** when

- it was properly submitted,
- it is marked **60%** or more, and
- it was **defended** in a lab.

Passing the Course: Necessary Conditions

All of these must hold, and none of them can be traded for marks elsewhere.

- **60%** or more for the course overall
- at least **11** in-lab worksheets ✔ **done**
- at least **6** problem sets ✔ **done**
- at least **1** problem set ✔ **done** and **defended**
- all **3** mini projects ✔ **done** (and **defended**)
- **60%** or more over the lecture quizzes

Passing the Course: Necessary Conditions

All of these must hold, and none of them can be traded for marks elsewhere.

- **60%** or more for the course overall
- at least **11** in-lab worksheets ✔ **done**
- at least **6** problem sets ✔ **done**
- at least **1** problem set ✔ **done** and **defended**
- all **3** mini projects ✔ **done** (and **defended**)
- **60%** or more over the lecture quizzes

Why it is built this way

> These are the parts of the course we watch you do.
> A grade earned without them would say nothing about you.

The In-Lab Component

- A **worksheet**, on paper, on the lecture you have just had, marked in the room against the key.
- Names are drawn, and those students **defend** their current problem set: explain your own code, then change it on request.

The In-Lab Component

- A **worksheet**, on paper, on the lecture you have just had, marked in the room against the key.
- Names are drawn, and those students **defend** their current problem set: explain your own code, then change it on request.

Mini projects are defended once each, over the three labs after the deadline. A failure may be retaken **once**, capped at a pass (i.e. 60%).

The In-Lab Component

- A **worksheet**, on paper, on the lecture you have just had, marked in the room against the key.
- Names are drawn, and those students **defend** their current problem set: explain your own code, then change it on request.

Mini projects are defended once each, over the three labs after the deadline. A failure may be retaken **once**, capped at a pass (i.e. 60%).

Come to the labs

> Neither can be made up afterwards, and the draw is random and repeats. A problem set you cannot defend scores **zero**, whatever you wrote.

Working with AI in This Course

Every exercise carries one of three marks, not the sheet it is on.

- **No AI.** Worksheets, lecture quizzes, the oral exam, and individual exercises inside a problem set. No devices, no assistant.
- **AI allowed.** You may use an assistant, and you may equally well not. Most problem set exercises.
- **AI recommended.** You are expected to use one, because the exercise is about directing and auditing it. The mini projects, and their audit sections.

Working with AI in This Course

Every exercise carries one of three marks, not the sheet it is on.

- **No AI.** Worksheets, lecture quizzes, the oral exam, and individual exercises inside a problem set. No devices, no assistant.
- **AI allowed.** You may use an assistant, and you may equally well not. Most problem set exercises.
- **AI recommended.** You are expected to use one, because the exercise is about directing and auditing it. The mini projects, and their audit sections.

Pitfall

Answering a **No AI** exercise with an assistant is plagiarism.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

1. **Hand in the transcript**, the full chat history, as a file next to your solution.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

1. **Hand in the transcript**, the full chat history, as a file next to your solution.
2. **Hand in the record**, a short report you write yourself: what you asked, what came back wrong, how you found it, and the test that catches it.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

1. **Hand in the transcript**, the full chat history, as a file next to your solution.
2. **Hand in the record**, a short report you write yourself: what you asked, what came back wrong, how you found it, and the test that catches it.
3. **Own every line.** If you cannot explain a line, or change it on request, it is not yours and it does not count.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

1. **Hand in the transcript**, the full chat history, as a file next to your solution.
2. **Hand in the record**, a short report you write yourself: what you asked, what came back wrong, how you found it, and the test that catches it.
3. **Own every line.** If you cannot explain a line, or change it on request, it is not yours and it does not count.
4. **Three parts, one mark.**
   The code earns it, the record justifies it, the defense confirms it.

Where AI is Allowed or Recommended

Four rules, and they are the same in both cases.

1. **Hand in the transcript**, the full chat history, as a file next to your solution.
2. **Hand in the record**, a short report you write yourself: what you asked, what came back wrong, how you found it, and the test that catches it.
3. **Own every line.** If you cannot explain a line, or change it on request, it is not yours and it does not count.
4. **Three parts, one mark.**
   The code earns it, the record justifies it, the defense confirms it.

Nothing here is a trick.
Using an assistant **where it is allowed** is not cheating.
Not being able to work without one is the actual risk.

Programming Paradigms Overview

What is a Programming Paradigm?



> *A **programming paradigm** is a relatively high-level way to conceptualize and structure the implementation of a computer program. A language can be classified as supporting one or more paradigms.*
>
> — Wikipedia (`https://en.wikipedia.org/wiki/Programming_paradigm`)

What is a Programming Paradigm?

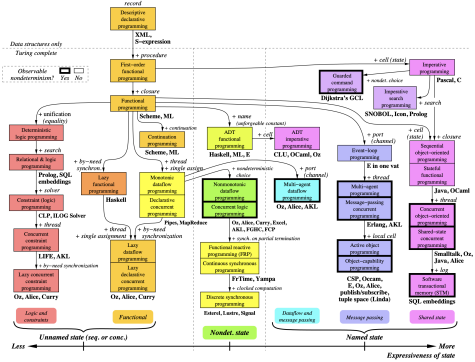
> *A **programming paradigm** is a relatively high-level way to conceptualize and structure the implementation of a computer program. A language can be classified as supporting one or more paradigms.*
>
> — Wikipedia (`https://en.wikipedia.org/wiki/Programming_paradigm`)

Programming paradigms are often classified into two large categories:

- **Imperative** — controls execution flow and state: *how* to compute;
- **Declarative** — code declares properties of the result: *what* to compute.

Taxonomy of Programming Paradigms

Programming paradigms are not defined strictly, and various taxonomies exist. For instance, taxonomy by Peter Van Roy[5] is often cited:



[5] Peter Van Roy et al. (2009). **"Programming paradigms for dummies: What every programmer should know".** In: *New computational paradigms for computer music* 104.

One Task, Said Two Ways

The query

Report the **three cheapest items still in stock**, with suppliers.

One Task, Said Two Ways

The query

> Report the **three cheapest items still in stock**, with suppliers.

- The items are in one table, the suppliers in another.

One Task, Said Two Ways

The query

Report the **three cheapest items still in stock**, with suppliers.

- The items are in one table, the suppliers in another.
- A supplier lookup costs something: a disk read, a round trip.

One Task, Said Two Ways

The query

> Report the **three cheapest items still in stock**, with suppliers.

- The items are in one table, the suppliers in another.
- A supplier lookup costs something: a disk read, a round trip.

Both return the same three rows.
Watch instead **how many lookups** each does.

Saying *How*: One Lookup per Item in Stock



```
result = []
for item in items:
    if item.in_stock:
        supplier = lookup_supplier(item.supplier_id)
        result.append((item.price, item.name, supplier.name))
result.sort()
return result[:3]
```

Saying *How*: One Lookup per Item in Stock



```
result = []
for item in items:
    if item.in_stock:
        supplier = lookup_supplier(item.supplier_id)
        result.append((item.price, item.name, supplier.name))
result.sort()
return result[:3]
```

This is **imperative**: the order of the work is written down and cannot change.

Saying *How*: One Lookup per Item in Stock

```
result = []
for item in items:
    if item.in_stock:
        supplier = lookup_supplier(item.supplier_id)
        result.append((item.price, item.name, supplier.name))
result.sort()
return result[:3]
```

This is **imperative**: the order of the work is written down and cannot change.

Ten thousand items in stock means ten thousand lookups.

Saying *What*: Three Lookups



```sql
select item.name, supplier.name
from item join supplier on item.supplier_id = supplier.id
where item.in_stock
order by item.price
limit 3
```

Saying *What*: Three Lookups



```sql
select item.name, supplier.name
from item join supplier on item.supplier_id = supplier.id
where item.in_stock
order by item.price
limit 3
```

This is **declarative**: it says which rows are wanted, not in what order.

Saying *What*: Three Lookups



```sql
select item.name, supplier.name
from item join supplier on item.supplier_id = supplier.id
where item.in_stock
order by item.price
limit 3
```

This is **declarative**: it says which rows are wanted, not in what order.

So the planner may cut to three *first*, and look up three suppliers after that.

Saying *What*: Three Lookups

```
select item.name, supplier.name
from item join supplier on item.supplier_id = supplier.id
where item.in_stock
order by item.price
limit 3
```

This is **declarative**: it says which rows are wanted, not in what order.

So the planner may cut to three *first*, and look up three suppliers after that.

The asymmetry

> Nothing stops you writing the loop the clever way.
> The query cannot be written the slow way.

The Same Query in This Course's Languages



Racket

```
(map with-supplier
  (take
    (sort (filter in-stock? items)
          < #:key price)
    3))
```

Haskell

```
map withSupplier
  (take 3
    (sortOn price
      (filter inStock items)))
```

The Same Query in This Course's Languages



Racket

```
(map with-supplier
  (take
    (sort (filter in-stock? items)
          < #:key price)
    3))
```

Haskell

```
map withSupplier
  (take 3
    (sortOn price
      (filter inStock items)))
```

In both, the lookup is written after the cut, so it happens after the cut: three lookups, not ten thousand.

The Same Query in This Course's Languages



Racket

```
(map with-supplier
  (take
    (sort (filter in-stock? items)
          < #:key price)
    3))
```

Haskell

```
map withSupplier
  (take 3
    (sortOn price
      (filter inStock items)))
```

In both, the lookup is written after the cut, so it happens after the cut: three lookups, not ten thousand.

In Haskell `sortOn` is lazy, so `take 3` stops it early too.
More on that in topic 9.

The Same Query in This Course's Languages

Racket

```
(map with-supplier
  (take
    (sort (filter in-stock? items)
          < #:key price)
    3))
```

Haskell

```
map withSupplier
  (take 3
    (sortOn price
      (filter inStock items)))
```

In both, the lookup is written after the cut, so it happens after the cut: three lookups, not ten thousand.

In Haskell `sortOn` is lazy, so `take 3` stops it early too.
More on that in topic 9.

Laziness goes further: in Haskell the `map` may come **first** and still cost three lookups, because a supplier is fetched only when something asks for it.
In Racket that same order costs ten thousand.

What the Two Words Actually Mean

- **Imperative** — the program is a sequence of steps, so you chose the order and you own the cost.

What the Two Words Actually Mean

- **Imperative** — the program is a sequence of steps, so you chose the order and you own the cost.
- **Declarative** — the program is a description of the answer. Something else chooses the order, and may choose better than you would.

What the Two Words Actually Mean

- **Imperative** — the program is a sequence of steps, so you chose the order and you own the cost.
- **Declarative** — the program is a description of the answer. Something else chooses the order, and may choose better than you would.

Neither is a property of the language.
Python has both, and so does Haskell. You see one of each in today’s lab.

What the Two Words Actually Mean

- **Imperative** — the program is a sequence of steps, so you chose the order and you own the cost.
- **Declarative** — the program is a description of the answer. Something else chooses the order, and may choose better than you would.

Neither is a property of the language.
Python has both, and so does Haskell. You see one of each in today's lab.

Question

> Which of the two do you write more of, and which do you review harder?

Common Programming Paradigms



1. **Imperative** — control execution flow and state
2. **Declarative** — describe properties of the desired result
3. **Structural** — clean `goto`-free nested control structures
4. **Procedural** — imperative programming with procedure calls
5. **Object-oriented** — objects with mutable internal state and public interfaces
6. **Functional** — function calls without (global) state
7. **Logic** — programming with facts and rules, relying on inference engine
8. **Relational** — programming with logical relations
9. **Differentiable** — programming with smooth manifolds
10. **Array** — wholemeal programming with data parallel structures
11. . . .

See more at `https://cs.lmu.edu/~ray/notes/paradigms/`.

Programming Paradigms and Concepts in This Course

Programming Paradigms in This Course

You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state

2. **Logic** — programming with logical relations

Programming Paradigms in This Course

You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming
   - **Statically typed** functional programming
2. **Logic** — programming with logical relations

Programming Paradigms in This Course



You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming in Racket
   - **Statically typed** functional programming
2. **Logic** — programming with logical relations

Programming Paradigms in This Course



You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming in Racket
   - **Statically typed** functional programming in Haskell
2. **Logic** — programming with logical relations

Programming Paradigms in This Course



You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming in Racket
   - **Statically typed** functional programming in Haskell
2. **Logic** — programming with logical relations in miniKanren

Programming Paradigms in This Course



You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming in Racket
   - **Statically typed** functional programming in Haskell
2. **Logic** — programming with logical relations in miniKanren

We will see both **Imperative** and **Declarative** examples in these languages.

Programming Paradigms in This Course



You already know imperative and object-oriented languages.
So we will focus on the paradigms you have seen least:

1. **Functional** — function calls without (global) state
   - **Dynamically typed** functional programming in Racket
   - **Statically typed** functional programming in Haskell
2. **Logic** — programming with logical relations in miniKanren

We will see both **Imperative** and **Declarative** examples in these languages.

For each paradigm, we will first study its concepts in a pure setting, then briefly discuss how these are used in popular multiparadigm languages.

Concepts in This Course

We will see the following concepts in this course[6]:

- pure functions
- immutable data structures
- tail recursion and local state
- higher-order functions
- parametric polymorphism
- algebraic data types
- exhaustive pattern matching
- accumulator parameter pattern
- effect systems
- lazy evaluation
- wholemeal programming
- list comprehensions
- relational programs
- modes of execution for relations

[6] The list of concepts might be incomplete.

About Functional Programming

Why functional programming matters? — Hughes, 1989

. . . Conventional languages place conceptual limits on the way problems can be modularised. Functional languages push those limits back. . . . Since modularity is the key to successful programming, functional languages are vitally important to the real world.

Why functional programming matters? — Hughes, 1989

. . . Conventional languages place conceptual limits on the way problems can be modularised. Functional languages push those limits back. . . . Since modularity is the key to successful programming, functional languages are vitally important to the real world.

How functional programming mattered — Hu, Hughes, and Wang, 2015

. . . Has functional programming really mattered? Our answer is a resounding 'Yes!'. . . .

How Functional Programming Matters

Here are a few key points and areas where FP has contributed a lot:

- Equational reasoning (a.k.a. substitution model in LISP)
- Property-based testing
- User-defined optimizations via rewrite rules
- Type classes (traits) vs. interfaces
- Types and logic
- Embedded DSLs
- Parallel, concurrent, and distributive programming

See “How functional programming mattered”[7].

[7] Zhenjiang Hu, John Hughes, and Meng Wang (July 2015). **“How functional programming mattered”.** In: *National Science Review* 2.3.

Functional Features in Other Languages



You have probably seen or used functional programming already:

- `map`, `filter`, and `fold` in most modern languages
- **lambda expressions** in Java, Python, C++, C#, Rust, and Dart; **function literals** in Go
- **list comprehensions** in Python; **for comprehensions** in Scala; **LINQ** in C#
- **algebraic datatypes**: **enums** in Rust, **cases** in Swift, **sealed interfaces** in Java, **sealed classes** in Kotlin and Dart
- **pattern matching** in Rust, Scala, Swift, Python, Java, and Dart
- **generics** in Java, C#, Rust, Go, Dart, and TypeScript

5 min break



Introduction to λ-calculus

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

Working with pure λ-calculus brings into focus some important ideas:

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

Working with pure λ-calculus brings into focus some important ideas:

1. pure (higher-order) functions are expressive: λ-calculus is Turing-complete

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

Working with pure λ-calculus brings into focus some important ideas:

1. pure (higher-order) functions are expressive: λ-calculus is Turing-complete
2. data types and control structures can be encoded as functions, which can even lead to optimizations[8]

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

Working with pure λ-calculus brings into focus some important ideas:

1. pure (higher-order) functions are expressive: λ-calculus is Turing-complete
2. data types and control structures can be encoded as functions, which can even lead to optimizations[8]
3. correctness of programs can be shown through *equational reasoning*

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

Pure Functions and λ-calculus

We now look at the core concept of functional programming: **pure functions**.

We use the most minimalistic one: λ-calculus.

Working with pure λ-calculus brings into focus some important ideas:

1. pure (higher-order) functions are expressive: λ-calculus is Turing-complete
2. data types and control structures can be encoded as functions, which can even lead to optimizations[8]
3. correctness of programs can be shown through *equational reasoning*

It is also one of the main formalisms for studying high-level features of programming languages, type systems above all (Pierce 2002, §5).

[8] Janis Voigtländer (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg.

λ-calculus: Syntax

Untyped λ-calculus

Programs in (untyped) λ-calculus are called **terms** (or **λ-terms**):

1. a *variable* (e.g x, y, or z) is a term
2. an *application* $(t_1\ t_2)$ of term t_1 to term t_2 is a term
3. an *abstraction* $\lambda x.t$ is a term if t is a term (possibly referring to x)

It can also be extended with constants, primitive operations, and so on.[9]

[9] B.C. Pierce (2002). ***Types and Programming Languages***. The MIT Press. MIT Press, §5.2.

λ-calculus: Syntax

Untyped λ-calculus

Programs in (untyped) λ-calculus are called **terms** (or **λ-terms**):

1. a *variable* (e.g x, y, or z) is a term
2. an *application* $(t_1\ t_2)$ of term t_1 to term t_2 is a term
3. an *abstraction* $\lambda x.t$ is a term if t is a term (possibly referring to x)

It can also be extended with constants, primitive operations, and so on.[9]

Examples of λ-terms

1. identity: $\lambda x.x$
2. constant: $\lambda x.42$
3. addition: $\lambda x.\lambda y.x + y$
4. selecting second: $(((\lambda x.\lambda y.y)\ a)\ b)$
5. repeated application: $f\ (f\ (f\ (f\ (f\ x))))$
6. Y-combinator: $\lambda f.(\lambda x.(f(x\ x)))\ (\lambda x.(f(x\ x)))$

[9] B.C. Pierce (2002). ***Types and Programming Languages***. The MIT Press. MIT Press, §5.2.

λ-calculus: Syntax Conventions

The following conventions are commonly used[10]:

1. Application is left-associative:

$$s\ t\ u \quad \text{is the same as} \quad (s\ t)\ u$$
$$f\ (g\ x\ y)\ z \quad \text{is the same as} \quad (f\ ((g\ x)\ y))\ z$$

[10] There exist different systems of conventions, e.g. Krivine's notation for λ-calculus.

λ-calculus: Syntax Conventions

The following conventions are commonly used[10]:

1. Application is left-associative:

$$s\ t\ u \quad \text{is the same as} \quad (s\ t)\ u$$
$$f\ (g\ x\ y)\ z \quad \text{is the same as} \quad (f\ ((g\ x)\ y))\ z$$

2. λ-binder covers everything to the right (until the end of line or a closing parenthesis):

$$\lambda x.\lambda y.x\ y\ x \quad \text{is the same as} \quad \lambda x.(\lambda y.(x\ y)\ x)$$
$$\lambda x.(\lambda y.x)\ y\ x \quad \text{is the same as} \quad \lambda x.(((\lambda y.x)\ y)\ x)$$

[10] There exist different systems of conventions, e.g. Krivine's notation for λ-calculus.

λ-calculus: Scopes of Variables

In a λ-term, each variable occurrence is either **free** or **bound**.

1. A variable is **bound** if there is a λ-binder for this variable covering it.
2. A variable that is not bound, is called **free**.

A λ-term is called **closed** if it has no free variables.

λ-calculus: Scopes of Variables

In a λ-term, each variable occurrence is either **free** or **bound**.

1. A variable is **bound** if there is a λ-binder for this variable covering it.
2. A variable that is not bound, is called **free**.

A λ-term is called **closed** if it has no free variables.

Examples of free and bound variables

Free variables are **black**; bound variables are indexed by their binder:

1. identity: $\lambda x_1.x_1$
2. constant: $\lambda x_1.\mathbf{z}$
3. addition: $\lambda x_1.\lambda y_2.x_1 + y_2$
4. selecting second: $(((\lambda x_1.\lambda x_2.x_2)\ \mathbf{a})\ \mathbf{b})$
5. repeated application: $\mathbf{f}\ (\mathbf{f}\ (\mathbf{f}\ (\mathbf{f}\ \mathbf{x})))$
6. Y-combinator: $\lambda f_1.(\lambda x_2.(f_1(x_2\ x_2)))\ (\lambda x_3.(f_1(x_3\ x_3)))$

λ-calculus: Scopes of Variables

In a λ-term, each variable occurrence is either **free** or **bound**.

1. A variable is **bound** if there is a λ-binder for this variable covering it.
2. A variable that is not bound, is called **free**.

A λ-term is called **closed** if it has no free variables.

Examples of free and bound variables

Free variables are **black**; bound variables are indexed by their binder:

1. identity: $\lambda x_1.x_1$
2. constant: $\lambda x_1.\mathbf{z}$
3. addition: $\lambda x_1.\lambda y_2.x_1 + y_2$
4. selecting second: $(((\lambda x_1.\lambda x_2.x_2)\ \mathbf{a})\ \mathbf{b})$
5. repeated application: $\mathbf{f}\ (\mathbf{f}\ (\mathbf{f}\ (\mathbf{f}\ \mathbf{x})))$
6. Y-combinator: $\lambda f_1.(\lambda x_2.(f_1(x_2\ x_2)))\ (\lambda x_3.(f_1(x_3\ x_3)))$

Which of the terms above are closed?

λ-calculus: Operational Semantics

Evaluation of λ-terms is defined by how application of an abstraction works:

$$(\lambda \mathbf{x}.\mathbf{t})\ \mathbf{u} \quad \longrightarrow \quad [\mathbf{x} \mapsto \mathbf{u}]\mathbf{t}$$

This rule is called β-reduction, and in it

- **x** is a *metavariable* ranging over possible bound variables
- **t** and **u** are *metavariables* ranging over possible λ-terms
- $[\mathbf{x} \mapsto \mathbf{u}]\mathbf{t}$ means "replace (*substitute*) all occurrences of **x** with **u** in term **t**"

λ-calculus: Operational Semantics

Evaluation of λ-terms is defined by how application of an abstraction works:

$$(\lambda \mathbf{x}.\mathbf{t})\ \mathbf{u} \quad \longrightarrow \quad [\mathbf{x} \mapsto \mathbf{u}]\mathbf{t}$$

This rule is called β-reduction, and in it

- **x** is a *metavariable* ranging over possible bound variables
- **t** and **u** are *metavariables* ranging over possible λ-terms
- $[\mathbf{x} \mapsto \mathbf{u}]\mathbf{t}$ means "replace (*substitute*) all occurrences of **x** with **u** in term **t**"

Examples of β-reduction rule

$$\begin{aligned}
(\lambda x.x)\ y &\longrightarrow [x \mapsto y]x &&\equiv y \\
(\lambda y.\lambda x.y)\ (\lambda z.z) &\longrightarrow [y \mapsto (\lambda z.z)](\lambda x.y) &&\equiv \lambda x.\lambda z.z \\
(\lambda z.z\ (\lambda z.z))(u\ r) &\longrightarrow [z \mapsto u\ r](z\ (\lambda z.z)) &&\equiv u\ r\ (\lambda z.z)
\end{aligned}$$

λ-calculus: Reduction Strategies

It is possible that there are several places in a single λ-term, where a β-reduction is possible.
A subterm that can be β-reduced is called **redex** (or **β-redex**).

1. In **full β-reduction** any redex can be reduced at any time, e.g.:

$$\text{id}\ (\text{id}\ (\lambda z.\underline{\text{id}\ z})) \longrightarrow \underline{\text{id}\ (\text{id}\ (\lambda z.z))} \longrightarrow \underline{\text{id}\ (\lambda z.z)} \longrightarrow \lambda z.z$$

λ-calculus: Reduction Strategies

It is possible that there are several places in a single λ-term, where a β-reduction is possible.
A subterm that can be β-reduced is called **redex** (or **β-redex**).

1. In **full β-reduction** any redex can be reduced at any time, e.g.:

$$\text{id}\ (\text{id}\ (\lambda z.\underline{\text{id}\ z})) \longrightarrow \underline{\text{id}\ (\text{id}\ (\lambda z.z))} \longrightarrow \underline{\text{id}\ (\lambda z.z)} \longrightarrow \lambda z.z$$

2. In **normal order** leftmost outermost redex is reduced first:

$$\underline{\text{id}\ (\text{id}\ (\lambda z.\text{id}\ z))} \longrightarrow \underline{\text{id}\ (\lambda z.\text{id}\ z)} \longrightarrow \lambda z.\underline{\text{id}\ z} \longrightarrow \lambda z.z$$

λ-calculus: Reduction Strategies

It is possible that there are several places in a single λ-term, where a β-reduction is possible.
A subterm that can be β-reduced is called **redex** (or **β-redex**).

1. In **full β-reduction** any redex can be reduced at any time, e.g.:

$$\mathrm{id}\ (\mathrm{id}\ (\lambda z.\underline{\mathrm{id}\ z})) \longrightarrow \underline{\mathrm{id}\ (\mathrm{id}\ (\lambda z.z))} \longrightarrow \underline{\mathrm{id}\ (\lambda z.z)} \longrightarrow \lambda z.z$$

2. In **normal order** leftmost outermost redex is reduced first:

$$\underline{\mathrm{id}\ (\mathrm{id}\ (\lambda z.\mathrm{id}\ z))} \longrightarrow \underline{\mathrm{id}\ (\lambda z.\mathrm{id}\ z)} \longrightarrow \lambda z.\underline{\mathrm{id}\ z} \longrightarrow \lambda z.z$$

3. In **applicative order** outermost redex and arguments are reduced first:

$$\mathrm{id}\ (\mathrm{id}\ (\lambda z.\underline{\mathrm{id}\ z})) \longrightarrow \mathrm{id}\ (\underline{\mathrm{id}\ (\lambda z.z)}) \longrightarrow \underline{\mathrm{id}\ (\lambda z.z)} \longrightarrow \lambda z.z$$

4. In **call-by-name**, leftmost outermost redex first, no reductions under λ-binder:

$$\underline{\mathsf{id}\ (\mathsf{id}\ (\lambda z.\mathsf{id}\ z))} \longrightarrow \underline{\mathsf{id}\ (\lambda z.\mathsf{id}\ z)} \longrightarrow \lambda z.\mathsf{id}\ z$$

λ-calculus: call-by-name and call-by-value

4. In **call-by-name**, leftmost outermost redex first, no reductions under λ-binder:

$$\underline{\mathsf{id}\ (\mathsf{id}\ (\lambda z.\mathsf{id}\ z))} \longrightarrow \underline{\mathsf{id}\ (\lambda z.\mathsf{id}\ z)} \longrightarrow \lambda z.\mathsf{id}\ z$$

5. In **call-by-value**, (outermost) arguments first, no reductions under λ-binder:

$$\mathsf{id}\ (\underline{\mathsf{id}\ (\lambda z.\mathsf{id}\ z)}) \longrightarrow \underline{\mathsf{id}\ ((\lambda z.\mathsf{id}\ z))} \longrightarrow \lambda z.\mathsf{id}\ z$$

λ-calculus: call-by-name and call-by-value

4. In **call-by-name**, leftmost outermost redex first, no reductions under λ-binder:

$$\underline{\mathsf{id}\ (\mathsf{id}\ (\lambda z.\mathsf{id}\ z))} \longrightarrow \underline{\mathsf{id}\ (\lambda z.\mathsf{id}\ z)} \longrightarrow \lambda z.\mathsf{id}\ z$$

5. In **call-by-value**, (outermost) arguments first, no reductions under λ-binder:

$$\mathsf{id}\ (\underline{\mathsf{id}\ (\lambda z.\mathsf{id}\ z)}) \longrightarrow \underline{\mathsf{id}\ ((\lambda z.\mathsf{id}\ z))} \longrightarrow \lambda z.\mathsf{id}\ z$$

If **call-by-name** and **call-by-value** terminate, they will always produce the same result. There are terms for which **call-by-name** terminates, but **call-by-value** does not.

λ-calculus: Substitution (attempt 1)

The following definition of substitution is **wrong**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \lambda y.[x \mapsto s]t_1 \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

λ-calculus: Substitution (attempt 1)

The following definition of substitution is **wrong**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \lambda y.[x \mapsto s]t_1 \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

Hint

How does $[x \mapsto f\ y](\lambda x.x)$ unfold?

λ-calculus: Substitution (attempt 2)

The following definition of substitution is **wrong**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \begin{cases} \lambda y.t_1, & \text{if } x = y \\ \lambda y.[x \mapsto s]t_1, & \text{otherwise} \end{cases} \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

λ-calculus: Substitution (attempt 2)

The following definition of substitution is **wrong**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \begin{cases} \lambda y.t_1, & \text{if } x = y \\ \lambda y.[x \mapsto s]t_1, & \text{otherwise} \end{cases} \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

Hint

How does $[x \mapsto f\ y](\lambda y.x)$ unfold?

λ-calculus: Substitution (attempt 3)

The following definition of substitution is **incomplete**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \begin{cases} \lambda y.t_1, & \text{if } x = y \\ \lambda y.[x \mapsto s]t_1, & \text{if } x \neq y \text{ and } y \notin FV(s) \end{cases} \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

λ-calculus: Substitution (attempt 3)

The following definition of substitution is **incomplete**. Why?

$$
\begin{aligned}
[x \mapsto s]x &= s \\
[x \mapsto s]y &= y \quad \text{if } x \neq y \\
[x \mapsto s](\lambda y.t_1) &= \begin{cases} \lambda y.t_1, & \text{if } x = y \\ \lambda y.[x \mapsto s]t_1, & \text{if } x \neq y \text{ and } y \notin FV(s) \end{cases} \\
[x \mapsto s](t_1\ t_2) &= [x \mapsto s]t_1\ [x \mapsto s]t_2
\end{aligned}
$$

Hint

What happens to $[x \mapsto f\ y](\lambda y.x)$?

λ-calculus: α-equivalence

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$	$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$	$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$	$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$	$(\lambda y.y)\ (\lambda x.x)$

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	is α-equivalent	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$		$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$		$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda y.y)\ (\lambda x.x)$

λ-calculus: α-equivalence

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	is α-equivalent	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$	is **not** α-equivalent	$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$		$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda y.y)\ (\lambda x.x)$

λ-calculus: α-equivalence

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	is α-equivalent	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$	is **not** α-equivalent	$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$	is α-equivalent	$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda y.y)\ (\lambda x.x)$

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	is α-equivalent	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$	is **not** α-equivalent	$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$	is α-equivalent	$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$	is α-equivalent	$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$		$(\lambda y.y)\ (\lambda x.x)$

Names of bound variables do not matter!

Terms t_1 and t_2 are called **α-equivalent**, if they can be produced from one another by renaming zero or more bound variables.

Examples and non-examples of α-equivalent terms

1. $(\lambda x.x)\ (\lambda x.x)$	is α-equivalent	$(\lambda y.y)\ (\lambda z.z)$
2. $\lambda x.y$	is **not** α-equivalent	$\lambda x.z$
3. $\lambda z.\lambda x.\lambda y.x\ (y\ z)$	is α-equivalent	$\lambda a.\lambda b.\lambda c.b\ (c\ a)$
4. $(\lambda x.y)\ (\lambda y.y)$	is α-equivalent	$(\lambda z.y)\ (\lambda x.x)$
5. $(\lambda x.y)\ (\lambda y.y)$	is **not** α-equivalent	$(\lambda y.y)\ (\lambda x.x)$

λ-calculus vs. Python (example 1)

Compare β-reduction of a λ-term

$$(\lambda x.\lambda y.x)\ 1\ 2 \quad \longrightarrow \quad (\lambda y.1)\ 2 \quad \longrightarrow \quad 1$$

With inlining of a Python program:

```python
def f(x):
  def g(y):
    return x
  return g

print(f(1)(2))
```

```python
# after inlining f

def g1(y):
  return 1

print(g1(2))
```

```python
# after inlining f
# and then inlining g1



print(1)
```

λ-calculus vs. Python (example 2)

Compare β-reduction of a λ-term with **free variables**

$$(\lambda x.\lambda y.x)\ y\ x \quad \longrightarrow \quad (\lambda z.y)\ x \quad \longrightarrow \quad y$$

With inlining of a Python program with **global variables**:

```
1  def f(x):
2    def g(y):
3      return x
4    return g
5
6  print(f(y)(x))
```

```
1  # after inlining f,
2  # with z renamed
3  def g1(z):
4    return y
5
6  print(g1(x))
```

```
1  # after inlining f
2  # and then inlining g1
3
4
5
6  print(y)
```

Quick recap

1. What does **imperative programming** control?
2. What does **declarative programming** describe instead?
3. Which concepts does the untyped λ**-calculus** bring into focus?
4. When are two λ-terms α**-equivalent**?

Quick recap

1. What does **imperative programming** control?
   The execution flow and the state, explicitly.
2. What does **declarative programming** describe instead?

3. Which concepts does the untyped λ**-calculus** bring into focus?

4. When are two λ-terms α**-equivalent**?

Quick recap

1. What does **imperative programming** control?
   The execution flow and the state, explicitly.
2. What does **declarative programming** describe instead?
   The properties of the desired result.
3. Which concepts does the untyped λ**-calculus** bring into focus?

4. When are two λ-terms α**-equivalent**?

Quick recap

1. What does **imperative programming** control?
   The execution flow and the state, explicitly.
2. What does **declarative programming** describe instead?
   The properties of the desired result.
3. Which concepts does the untyped λ**-calculus** bring into focus?
   Pure functions, higher-order functions, and equational reasoning.
4. When are two λ-terms α**-equivalent**?

Quick recap

1. What does **imperative programming** control?
   The execution flow and the state, explicitly.
2. What does **declarative programming** describe instead?
   The properties of the desired result.
3. Which concepts does the untyped λ**-calculus** bring into focus?
   Pure functions, higher-order functions, and equational reasoning.
4. When are two λ-terms α**-equivalent**?
   When renaming bound variables turns one into the other.

Quiz for Lecture 1

- Ten minutes, on Moodle, now. Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 1

- Ten minutes, on Moodle, now. Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

> Say what you learned and what stayed unclear, **specifically**.
> I read them before writing the next lecture.

What's Next

In Saturday's lab you will

1. learn about Church encoding for booleans, natural numbers, and pairs
2. practice writing and evaluating simple programs (terms) in λ-calculus

→ **Before next week**

Next week we start on Racket.

- Install DrRacket from `https://download.racket-lang.org`
- Read *"Quick: An Introduction to Racket with Pictures"* `https://docs.racket-lang.org/quick/index.html`
- Read *Racket Essentials 2.1–2.2* `https://docs.racket-lang.org/guide/to-scheme.html`

Thank you!



References

- Augustsson, Lennart et al. (2023). **"The Verse Calculus: a Core Calculus for Functional Logic Programming".** In: *Proceedings of the ACM on Programming Languages (ICFP)*. URL: https://simon.peytonjones.org/verse-calculus/.
- Hu, Zhenjiang, John Hughes, and Meng Wang (July 2015). **"How functional programming mattered".** In: *National Science Review* 2.3, pp. 349–370. ISSN: 2095-5138. DOI: 10.1093/nsr/nwv042. eprint: https://academic.oup.com/nsr/article-pdf/2/3/349/31566307/nwv042.pdf. URL: https://doi.org/10.1093/nsr/nwv042.

References ii

- Hughes, J. (Jan. 1989). **"Why Functional Programming Matters".** In: *The Computer Journal* 32.2, pp. 98–107. ISSN: 0010-4620. DOI: 10.1093/comjnl/32.2.98. eprint: https://academic.oup.com/comjnl/article-pdf/32/2/98/1445644/320098.pdf. URL: https://doi.org/10.1093/comjnl/32.2.98.
- Kudasov, Nikolai et al. (2024). ***Free Foil: Generating Efficient and Scope-Safe Abstract Syntax.*** arXiv: 2405.16384 [cs.PL]. URL: https://arxiv.org/abs/2405.16384.

References iii

- Lang, Nguyet-Anh H. et al. (2026). ***Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming.*** arXiv: 2601.02060 [cs.SE]. URL: https://arxiv.org/abs/2601.02060.
- Pierce, B.C. (2002). ***Types and Programming Languages.*** The MIT Press. MIT Press. ISBN: 9780262162098. URL: https://books.google.ru/books?id=ULT4DwAAQBAJ.
- Van Roy, Peter et al. (2009). **"Programming paradigms for dummies: What every programmer should know".** In: *New computational paradigms for computer music* 104, pp. 616–621.

References iv



Voigtländer, Janis (2008). **"Asymptotic Improvement of Computations over Free Monads".** In: *Mathematics of Program Construction*. Ed. by Philippe Audebaud and Christine Paulin-Mohring. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 388–403. ISBN: 978-3-540-70594-9.